

JSON Web Token (JWT) Fundamentals

Why JWTs?



Scott Brady

Identity Consultant & Engineering Manager

@scottbrady91

www.scottbrady91.com



Overview



What is a JWT?

What are JWTs used for?

How do you use a JWT?

When should you use a JWT?

What is JOSE?



What is a JWT?



A JSON Web Token (JWT)

eyJ0eXAiOiJhdCtqd3QiLCJhbGciOiJFUzI1NiIsImtpZCI6IjMxMzZjNWZhYWE0MmIwNz
hkZTZkNGEwYmQ0OGE2ZWZmIn0.eyJpc3MiOiJodHRwczovL2lkC5sb2NhbCI6ImF1ZCI6
ImFwaTEiLCJzdWIiOiI1YmU4NjM1OTA3M2M0MzRiYWQyZGEzOTMyMjIyZGF1ZSI6ImNsaW
VudF9pZCI6ImFwcCI6ImV4cCI6MTY1MzU3NzA1OiwiaWF0IjoxNjUzNTczNDU0LCJqdGki
OiIwZTc2NTkwY2NiN2RmYzY3ODZiZmU2NDEzYjk1MTg2OSJ9.vIviL0bafKHNXM1_5YY-
kkTw6gWD791WJGKz2BH0j9EU4MrUeyavhJoWwq11xX4mRTH1uqx4ECpziFyAhJjvwg

`eyJ = base64( { " )`

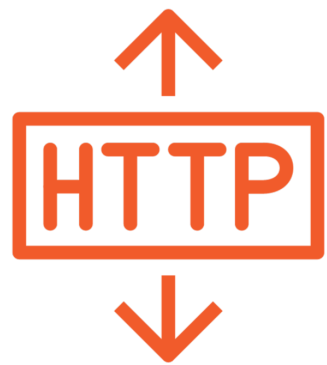
`A-Za-z0-9- _`



JSON Web Tokens (JWTs)



Standardized format



Securely transmit data between systems



Tamperproof (thanks to digital signatures)



JSON Web Tokens (JWTs)



Issuer

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6I1Njb3R0IiwiaWF0IjoxNjU0NTY2fQ.kCQyi0jI-nj_aMh5VTtqSt8vpNpcPxdj8PVd9FH5ByddZekMXxe43vn4N44Cgu208aW-y-Poq2j1EALHDof30Q
```

HTTP request + token



Recipient



```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6I1Njb3R0IiwiaWF0IjoxNjU0NTY2fQ.kCQyi0jI-nj_aMh5VTtqSt8vpNpcPxdj8PVd9FH5ByddZekMXxe43vn4N44Cgu208aW-y-Poq2j1EALHDof30Q
```



A JSON Web Token (JWT)

eyJ0eXAiOiJhdCtqd3QiLCJhbGciOiJFUzI1NiIsImtpZCI6IjMxMzZjNWZhYWE0MmIwNz
hkZTZkNGEwYmQ0OGE2ZWZmIn0.eyJpc3MiOiJodHRwczovL2lk
ImFwaTEiLCJzdWIiOiI1YmU4NjM1OTA3M2M0MzRiYWQyZGEz
OTMyMjIyZGF1ZSIsImNsaWVudF9pZCI6ImFwcCI6ImV4cCI6MTY1MzU3NzA1
OCwiaWF0IjoxNjUzNTczNDU4LCJqdGkiOiIwZTc2NTkwY2NiN2RmYzY3
ODZiZmU2NDEzYjk1MTg2OSJ9.vIviL0bafKHNXM1_5YY-k
kTw6gWD791WJGKz2BH0j9EU4MrUeyavhJoWwq1lxX4mRTH1uqx4ECpziFyAhJjvwg



```
{  
  "typ": "at+jwt",  
  "alg": "ES256",  
  "kid": "3136c5aaaa42b078de6d4a0bd48a6eff"  
}
```

Header

Describes the token itself

How to read & validate the token

```
{  
  "iss": "https://idp.local",  
  "aud": "api1",  
  "sub": "5be86359073c434bad2da3932222dabe",  
  "client_id": "app",  
  "exp": 1653577058,  
  "iat": 1653573458,  
  "jti": "0e76590ccb7dfc6786bfe6413b951869"  
}
```

Payload

The token's data as a JSON object

Contains claims about the entity

eyJ0eXAiOiJhdCtqd3QiLCJhbGciOiJFUzI1NiIsImtpZCI6IjMxMzZjNWFhYWE0MmIwNz
hkZTZkNGEwYmQ0OGE2ZWZmIn0.eyJpc3MiOiJodHRwczovL2lkcc5sb2NhbCI6ImF1ZCI6
ImFwaTEiLCJzdWIiOiI1YmU4NjM1OTA3M2M0MzRiYWQyZGEzOTMyMjIyZGF1ZSI6ImNsaW
VudF9pZCI6ImFwcCI6ImV4cCI6MTY1MzU3NzA1OiwiaWF0IjoxNjUzNTczNDU4LCJqdGki
OiIwZTc2NTkwY2NiN2RmYzY3ODZiZmU2NDEzYjk1MTg2OSJ9.vIviL0bafKHNXM1_5YY-k
kTw6gWD791WJGKz2BH0j9EU4MrUeyavhJoWwql1xX4mRTH1uqx4ECpziFyAhJjvwg

Signature

Created using the header, payload, and signing key

Base64url encoded signature value

Length varies based on algorithm and key

How to Pronounce JWT

“JSON Web Token”

“JWT”

“jot”

“joot!”

“ja-what?”



The JWT Use Case





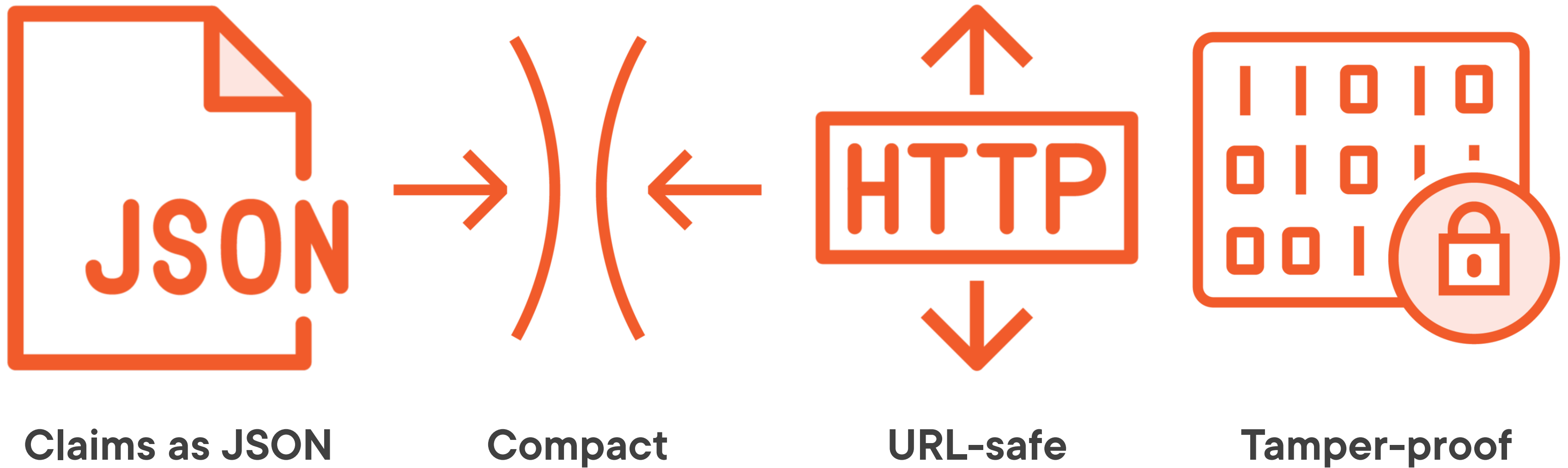
Originally created by the OAuth working group

Created due to demand for JSON representation of claims

- Use as self-contained OAuth access tokens
- Replace SAML assertions



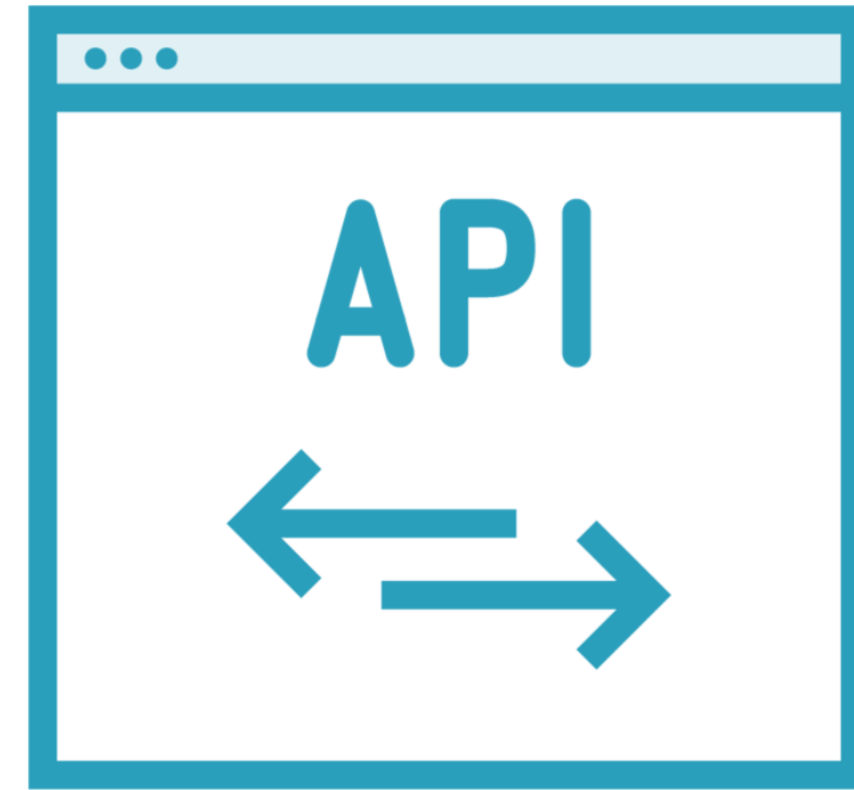
JWT Design



OAuth 2 Recap



Authorization framework

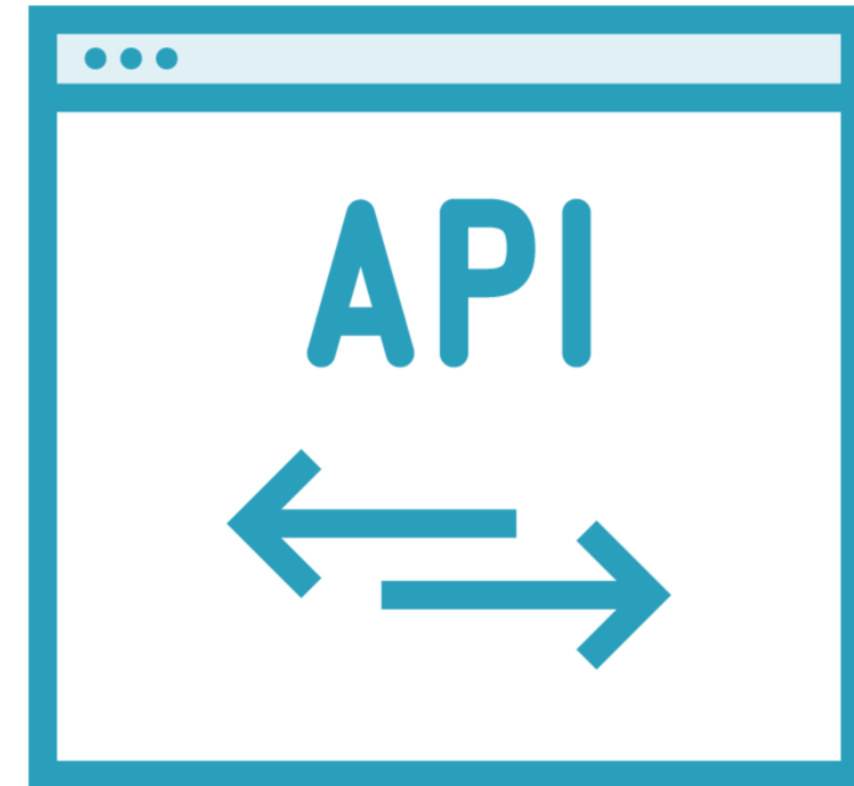


Built for HTTP APIs

OAuth 2 Recap



Delegation protocol



Built for HTTP APIs

OAuth 2 Delegation



Application

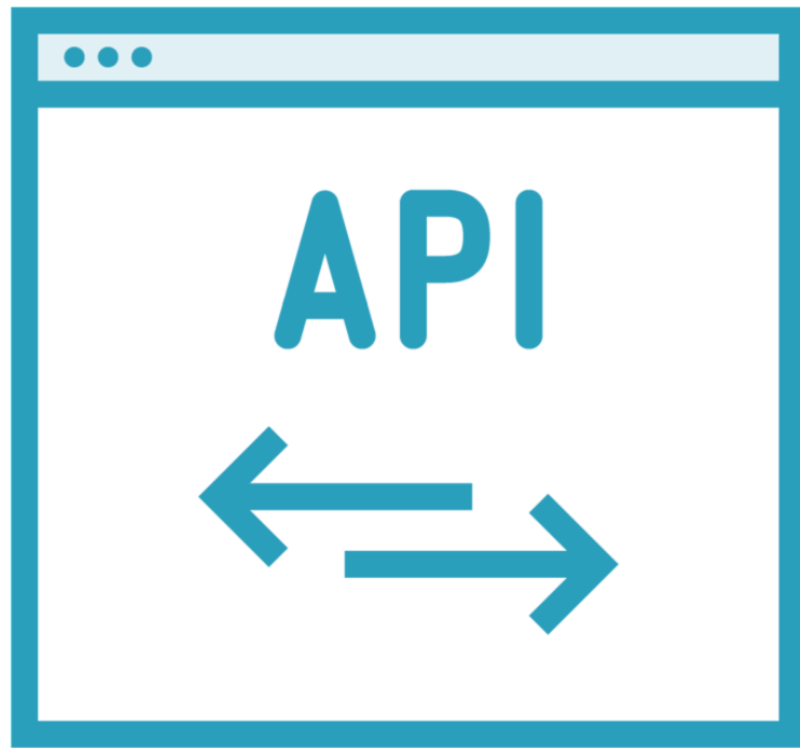
“Can I access this API to read and
send emails from your account?”

“Sure! But you can only
send emails.”



User

The OAuth 2 Players



**Protected
Resource
(HTTP API)**



**Client Application
(the requesting
application)**

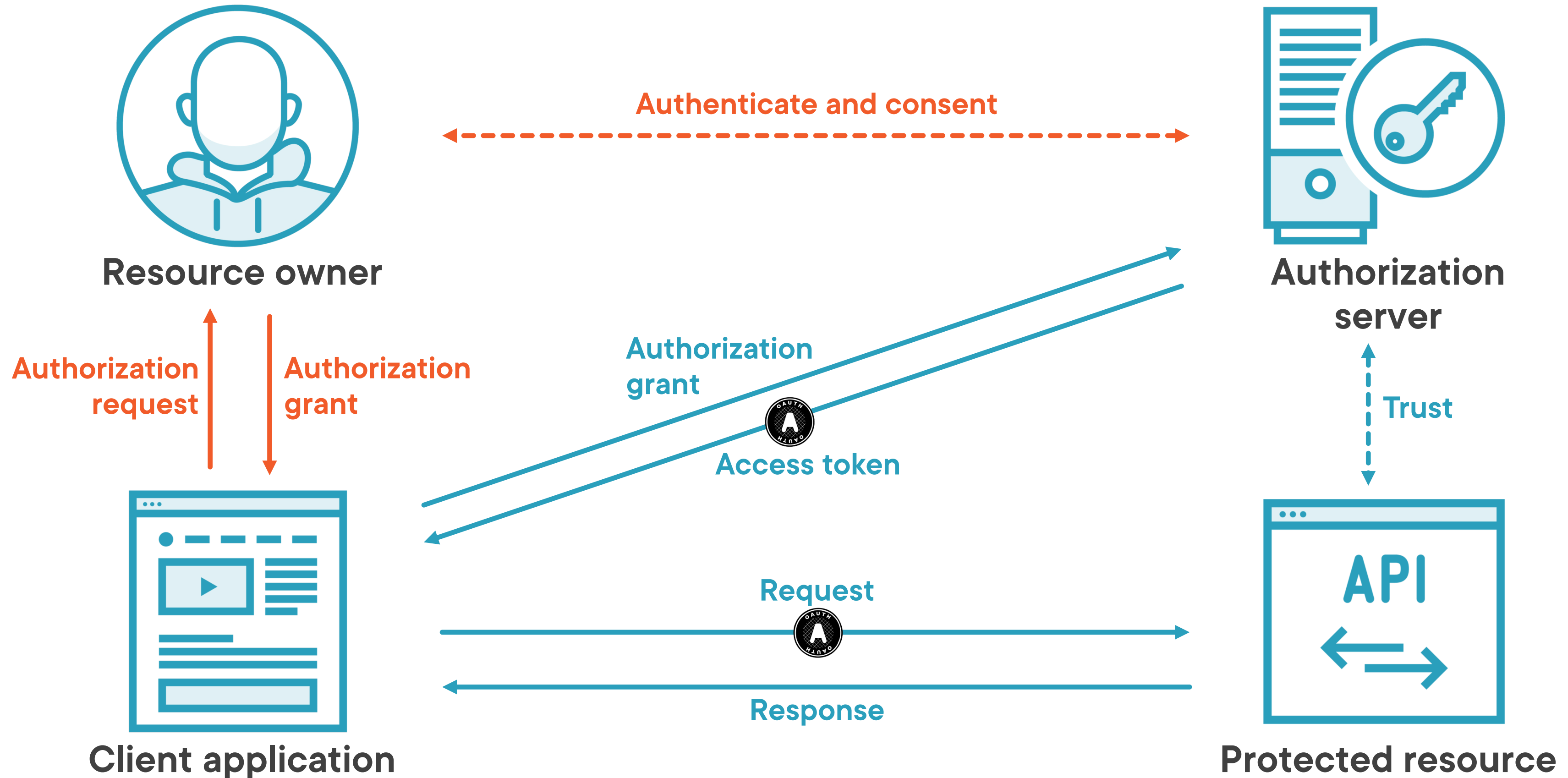


**Resource Owner
(the user)**



**Authorization server
(the token issuer)**

The OAuth Dance



OAuth Access Tokens



No defined format

No defined validation process



OAuth Access Tokens



Scopes (permissions granted by the user)



Token lifetime



Token subject (user or machine)



OAuth Access Token Example

HTTP/1.1 200 OK

Content-Type: application/json;charset=UTF-8

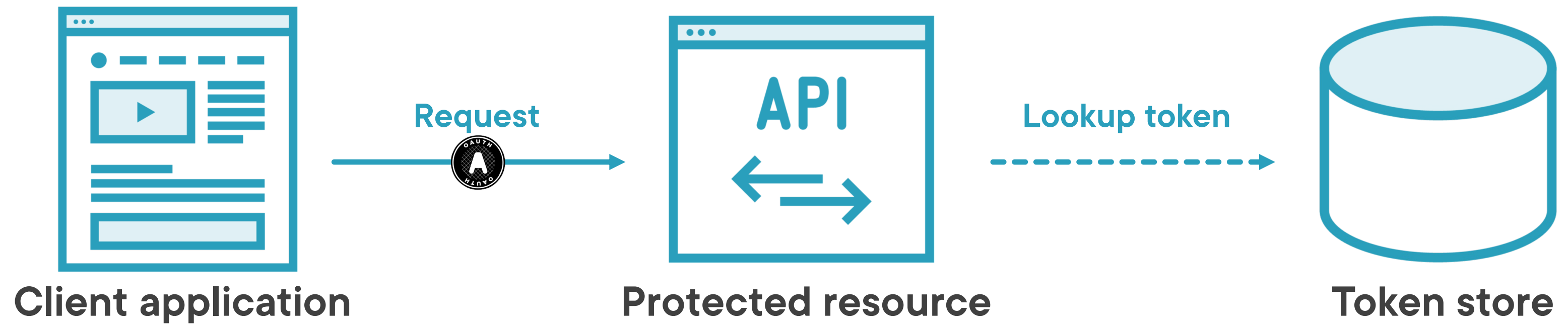
Cache-Control: no-store

Pragma: no-cache

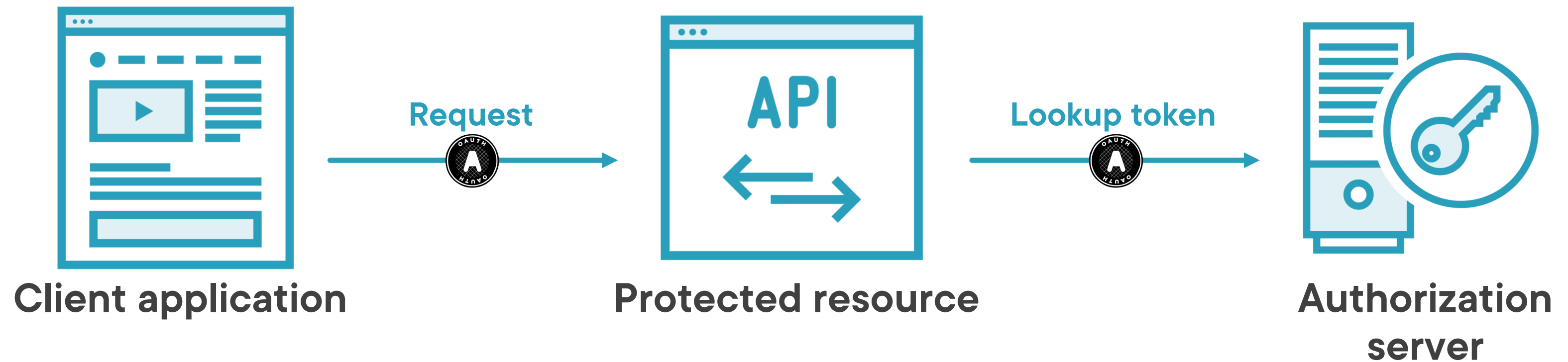
```
{  
  "access_token": "2YotnFZFEjr1zCsicMWpAA",  
  "token_type": "example",  
  "expires_in": 3600,  
  "refresh_token": "tGzv3J0kF0XG5Qx2TlKWIA",  
  "example_parameter": "example_value"  
}
```



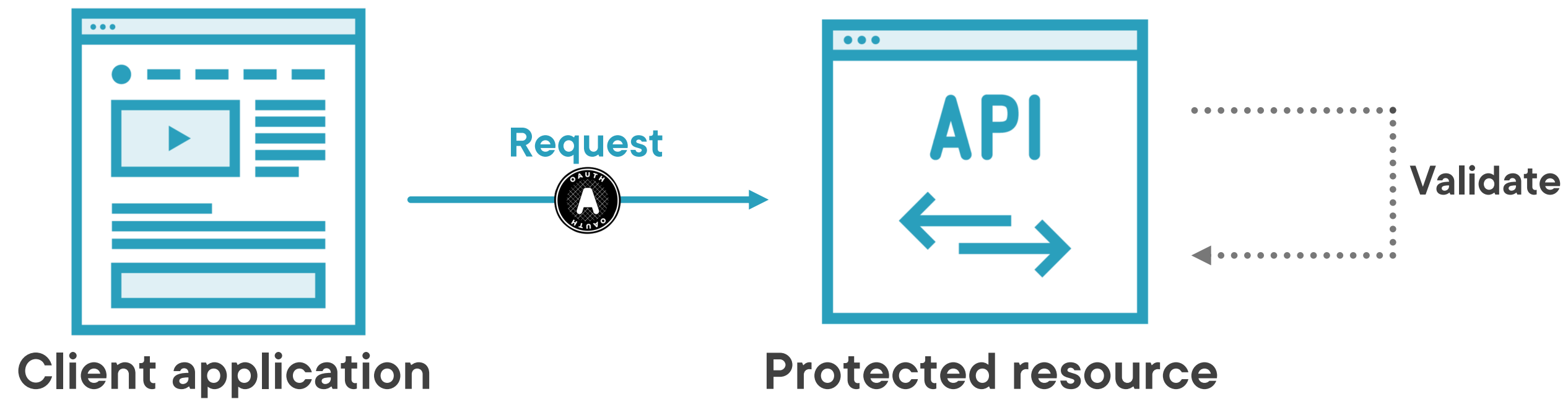
Access Token Validation



Access Token Validation



Access Token Validation



JWT Access Tokens



Self-contained tokens (stateless)



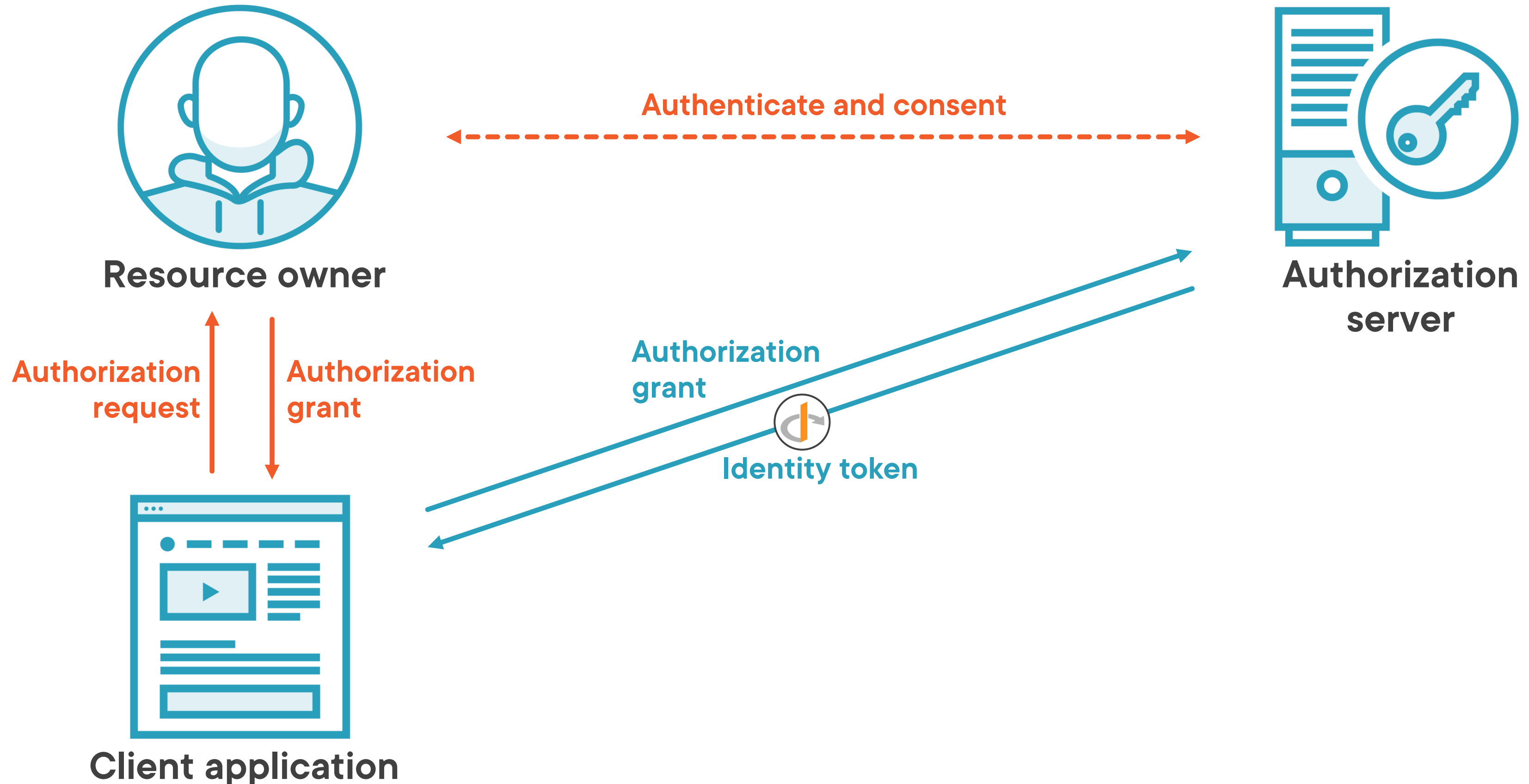
Can be validated without any extra hops



Now has a dedicated RFC



The OpenID Connect Dance



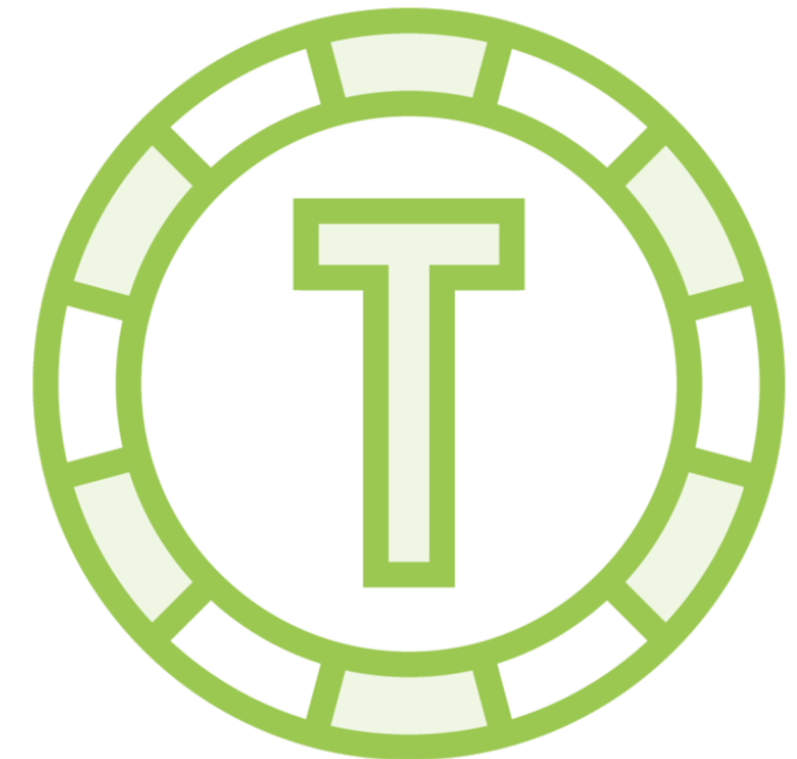
Identity Tokens



**Intended for the
client application
(never the API)**



**Describes how the
user authenticated**



Always a JWT

Identity Tokens vs SAML Assertions

Identity Token

Describes how the user authenticated

Can contain identity data

Digitally signed (JWS)

SAML Assertion

Describes how the user authenticated

Can contain identity data

Digitally signed (XMLDSig)



OpenID Connect Identity Token

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://sp.example.com",  
  "nbf": 1602927477,  
  "exp": 1602927777,  
  "iat": 1602927477,  
  "nonce": "e3f1c258a9a047bfb13a78fa6ace004a",  
  "sid": "472ae8c4776c4d109a9ed7ddb8780cfa",  
  "sub": "994211131",  
  "auth_time": 1602927277,  
  "amr": [ "pwd" ],  
  "given_name": "Scott",  
}
```



SAML Assertion

```
<saml2:Assertion ID="_af23066e-7f08-454a-9fe4-425c04d37aec" Version="2.0" IssueInstant="2020-10-17T09:24:25Z">
  <saml2:Issuer>https://idp.example.com</saml2:Issuer>
  <saml2:Subject>
    <saml2:NameID Format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified">994211131</saml2:NameID>
    <saml2:SubjectConfirmation Method="urn:oasis:names:tc:SAML:2.0:cm:bearer">
      <saml2:SubjectConfirmationData NotOnOrAfter="2020-10-17T09:26:25Z" InResponseTo="_e3f1c258a9a047bfb13a78fa6ace004a"
        Recipient="https://sp.local/saml/acs" />
    </saml2:SubjectConfirmation>
  </saml2:Subject>
  <saml2:Conditions NotBefore="2020-10-17T09:24:25Z" NotOnOrAfter="2020-10-17T09:26:25Z">
    <saml2:AudienceRestriction>
      <saml2:Audience>https://sp.example.com</saml2:Audience>
    </saml2:AudienceRestriction>
  </saml2:Conditions>
  <saml2:AttributeStatement>
    <saml2:Attribute Name="http://schemas.xmlsoap.org/ws/2005/05/identity/claims/givenname">
      <saml2:AttributeValue>Scott</saml2:AttributeValue>
    </saml2:Attribute>
  </saml2:AttributeStatement>
  <saml2:AuthnStatement AuthnInstant="2020-10-17T09:24:25Z" SessionIndex="_af23066e-7f08-454a-9fe4-425c04d37aec">
    <saml2:AuthnContext>
      <saml2:AuthnContextClassRef>urn:oasis:names:tc:SAML:2.0:ac:classes:Password</saml2:AuthnContextClassRef>
    </saml2:AuthnContext>
  </saml2:AuthnStatement>
</saml2:Assertion>
```





More Information

Getting Started with OAuth 2

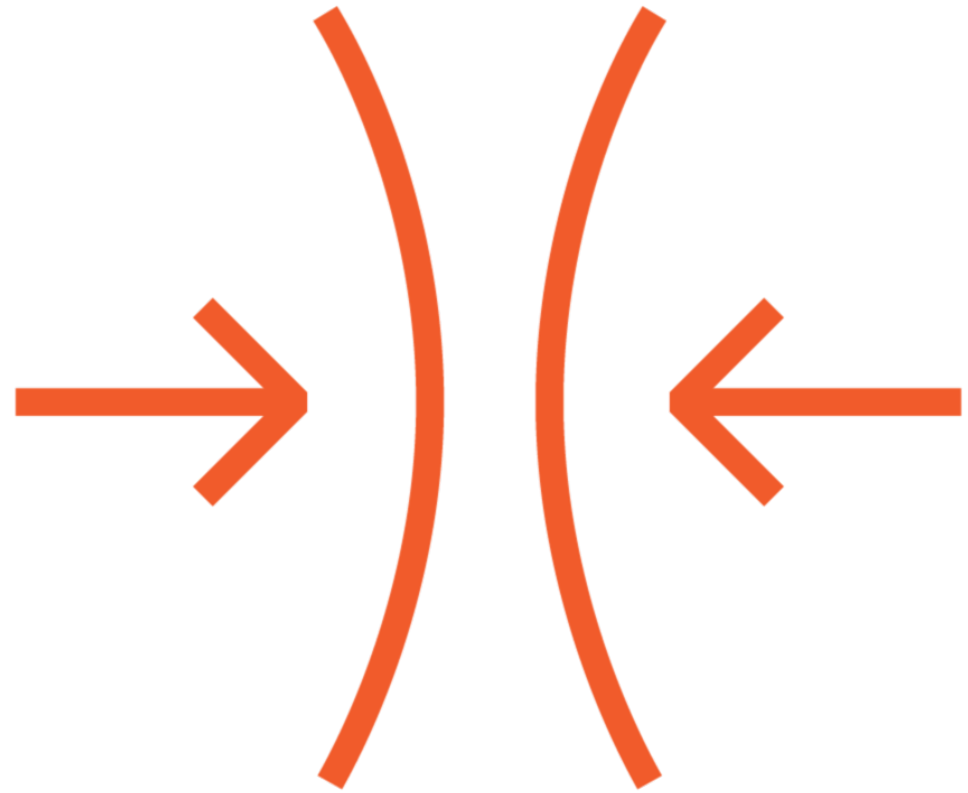
Scott Brady



How JWTs Are Typically Used



JWT Design



Compact



URL-safe



HTTP Requests

Path

Query string

Header

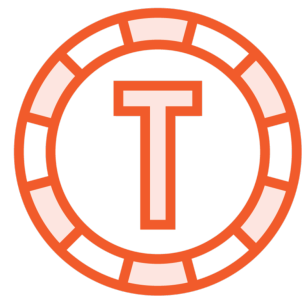
Body



JWTs for API Access



A JWT is a credential



Can be used as an OAuth bearer token



Must be kept a secret



Must use Transport Layer Security (TLS, previously SSL)



HTTP Requests

Path

Query string

Header

Body



JWT in the URL



**Path and query string
decrypted first**



**Often written to the
server logs**



**Plaintext JWTs written
to disk**

HTTP Requests

Path

Query string

Header

Body



Authorization Header

`Authorization: <authentication-scheme> <credential>`

`Authorization: Basic YWxhZGRpbjpvcGVuc2VzYW11`

`Authorization: Negotiate YIIVDAYGKwYBE`



Authorization Header

Bearer Scheme

Authorization: Bearer

eyJ0eXAiOiJhdCtqd3QiLCJhbGciOiJFUzI1NiIsImtpZCI6IjMxMzMzjNWFhYWE0MmIwNzhkZTZkNGEwY
mQ00GE2ZWZmIn0.eyJpc3MiOiJodHRwczoVL2lkC5sb2NhbiCI9ImF1ZCI6ImFwaTEiLCJzdWIiOiI1Ym
U4NmM1OTA3M2M0MzMzRiYWQyZGEzOTMyMjIyZGF1ZSI9ImNsaWVudF9pZCI6ImFwcCI9ImV4cCI6MTY1MzU
3NzA1OiwiaWF0IjoxNTczNDU4LCJqdGkiOiIwZTc2NTkwY2NiN2RmYzY3ODZiZmU2NDEzYjk1MTg2
OSJ9.vIviLObafKHNXM1_5YY-kkTw6gWD791WJGKz2BH0j9EU4MrUeyavhJoWwq1lxX4mRTHluqx4ECpz
iFyAhJjvwg

WWW-Authenticate Header

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Basic realm="Access to environment", charset="UTF-8"



WWW-Authenticate Header

Bearer Scheme

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Bearer realm="https://idp.example.com",
error="invalid_token",
error_description="The token has expired"



Authorization Policy Failure

HTTP/1.1 403 Forbidden



HTTP Requests

Header

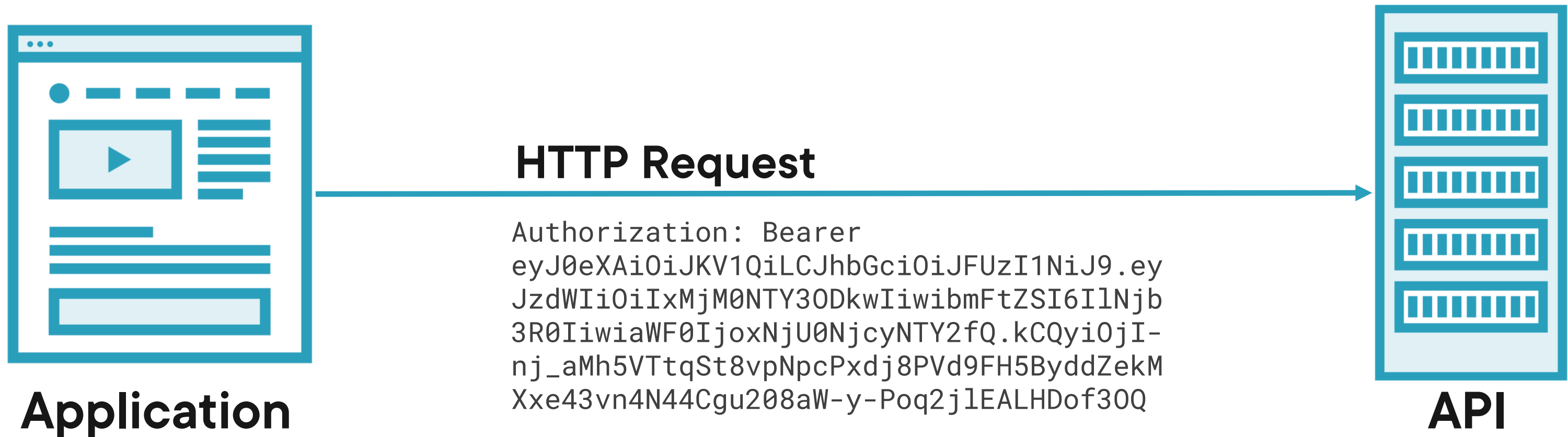
Body



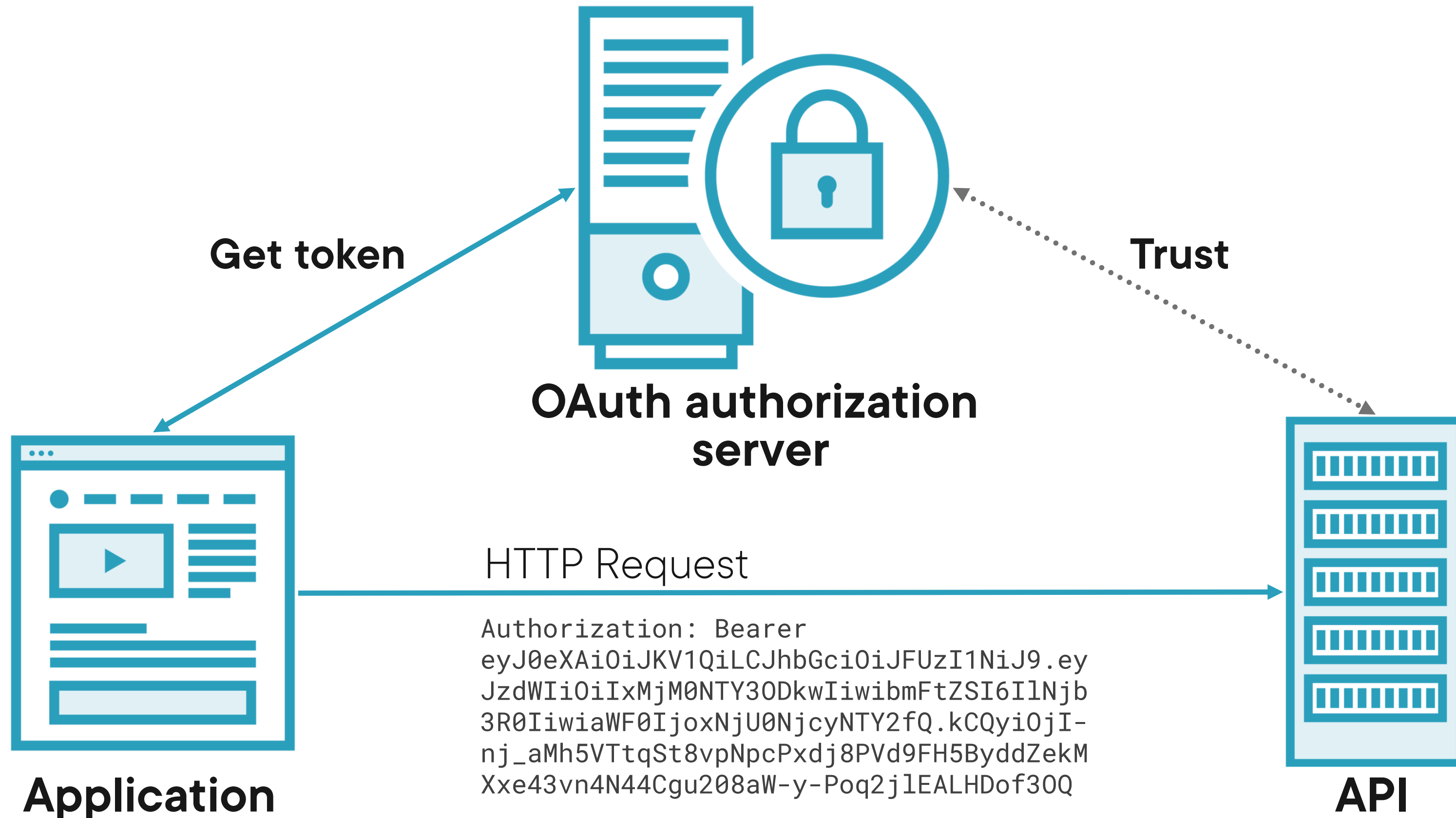
When to Use JWTs



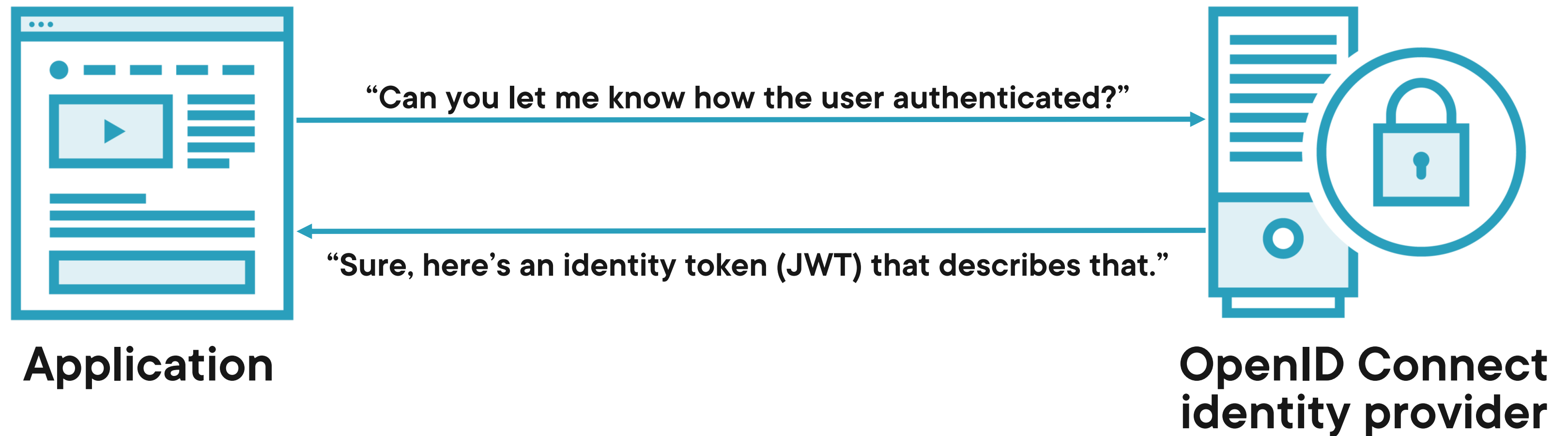
JWTs for API Access



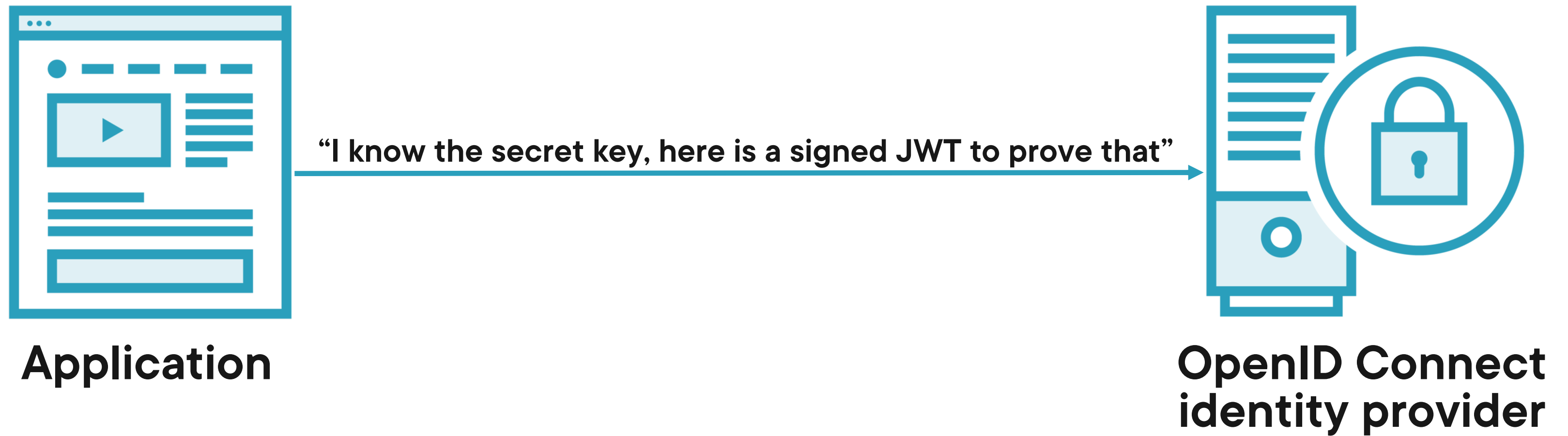
JWTs for API Access



JWTs for Information Transfer



JWTs as Security Proofs



Always use JWTs in
combination with something
else.



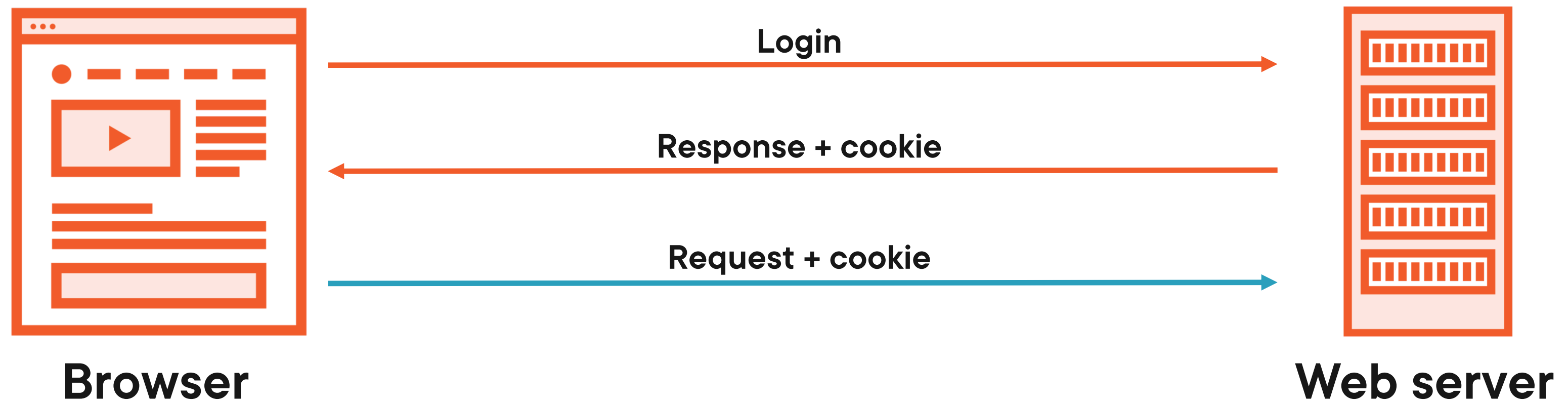
When Not to Use JWTs



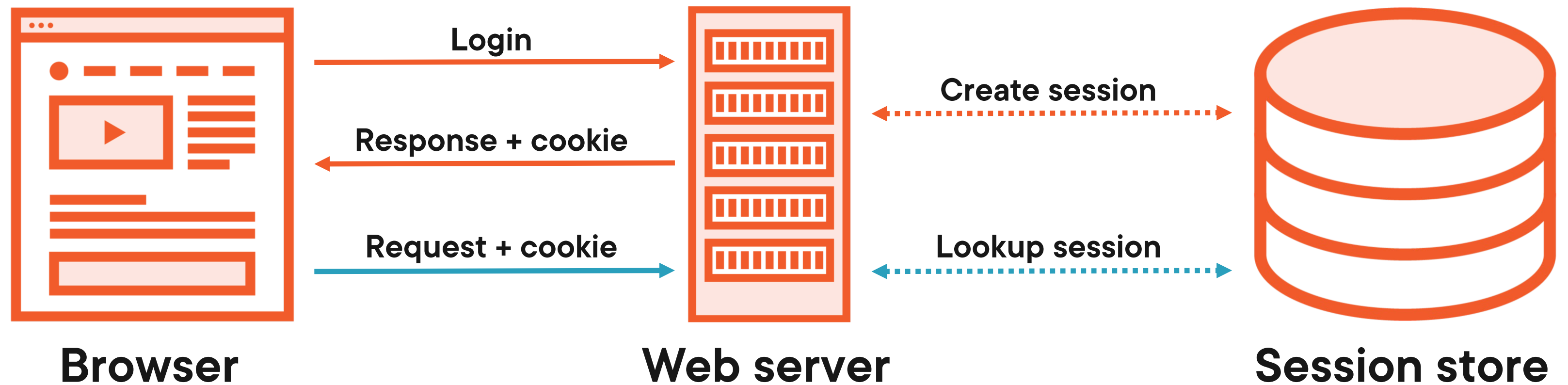
JWTs are not a replacement for
cookies and sessions.



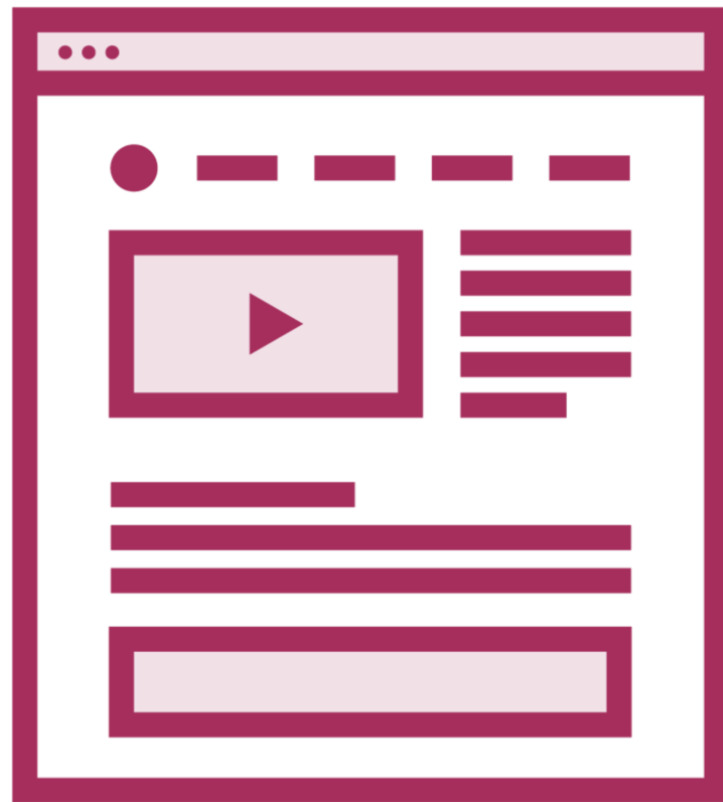
Traditional Web Session



Traditional Web Session



JWTs as Sessions



**Browser cannot
manage JWT
sessions**



**Have to
implement token
storage and
management**



**Can only use
storage visible
to JavaScript**



**No out-of-the-
box method to
invalidate a
single JWT**

Learn More About JWTs as Sessions



“Getting Started with OAuth” – Pluralsight



“JWT should not be your default for sessions” – Evert Pot



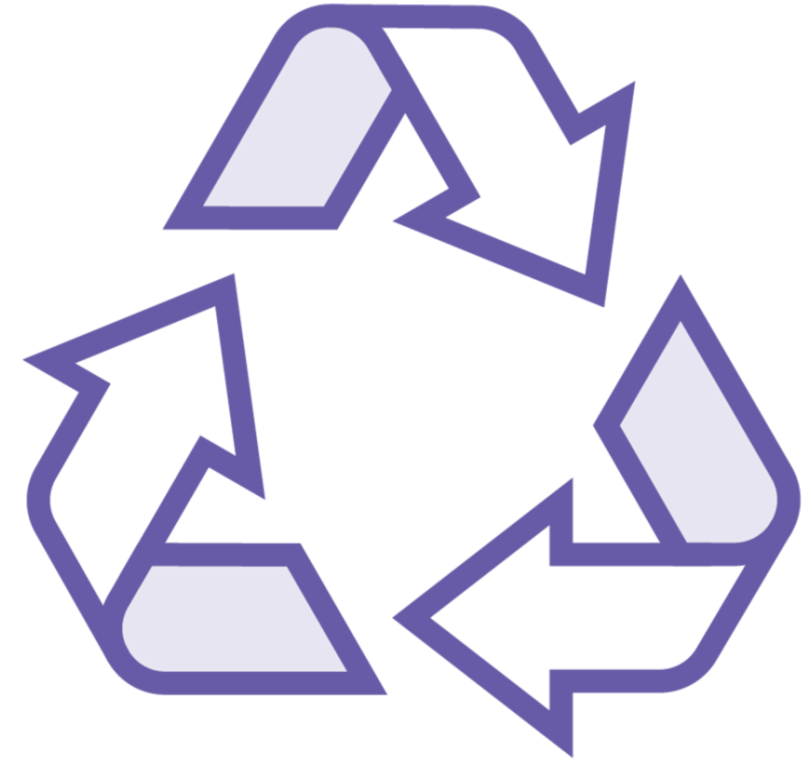
JWT Revocation



JWTs are valid until they expire



An individual JWT cannot be invalidated

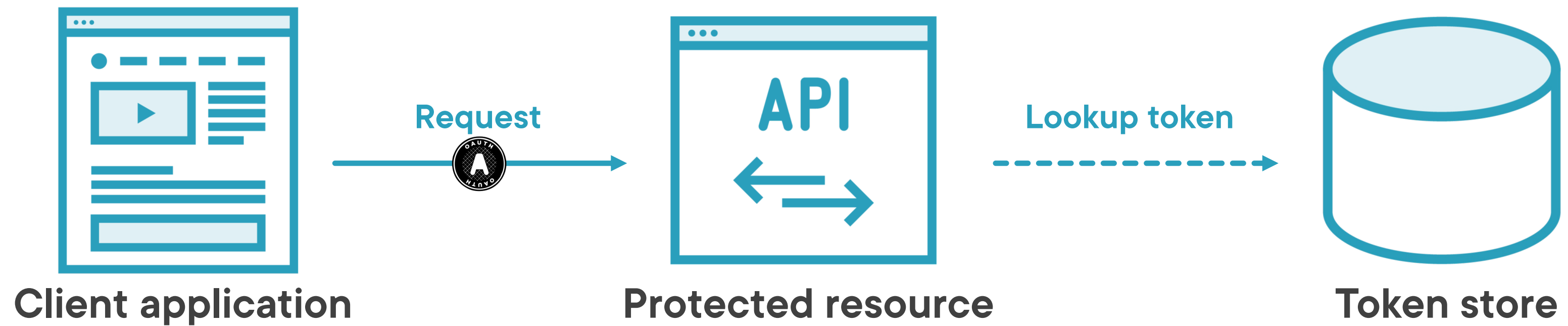


JWTs can be re-used

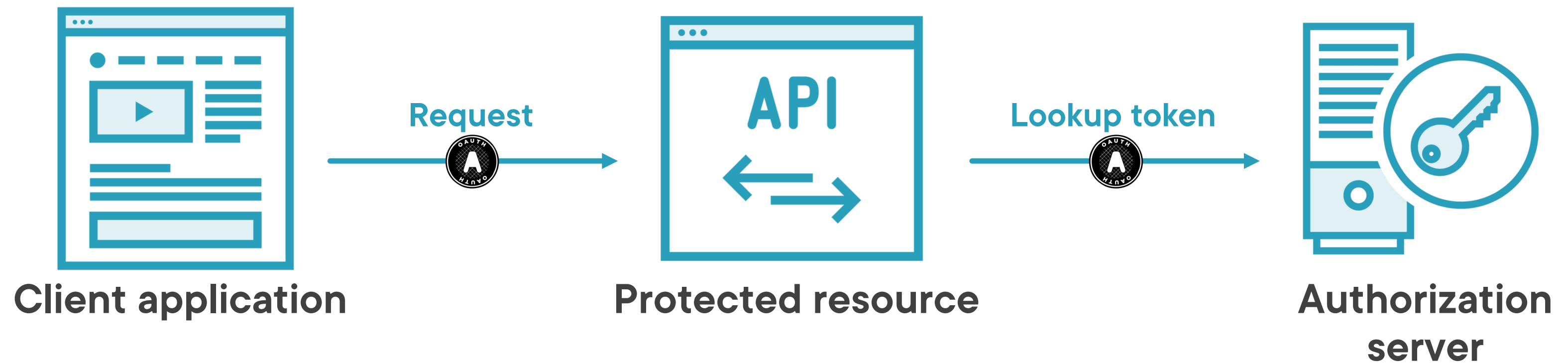
Consider stateful tokens
instead.



Stateful Tokens – Token Store



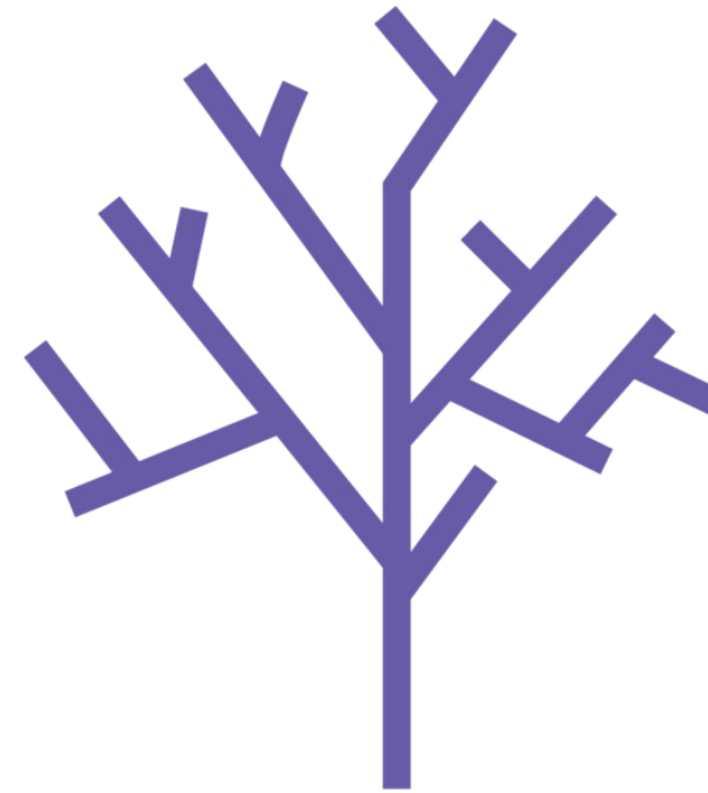
Stateful Tokens – Introspection



JWT Blocklists

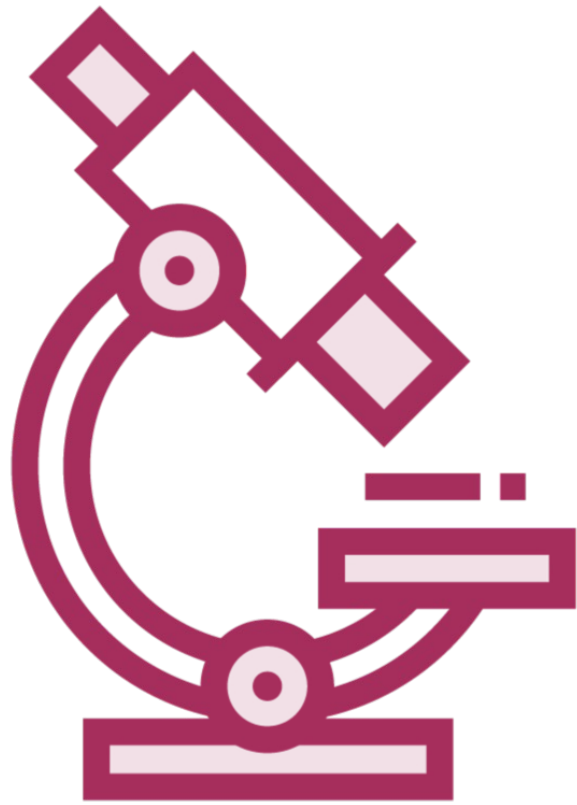


Makes scaling harder

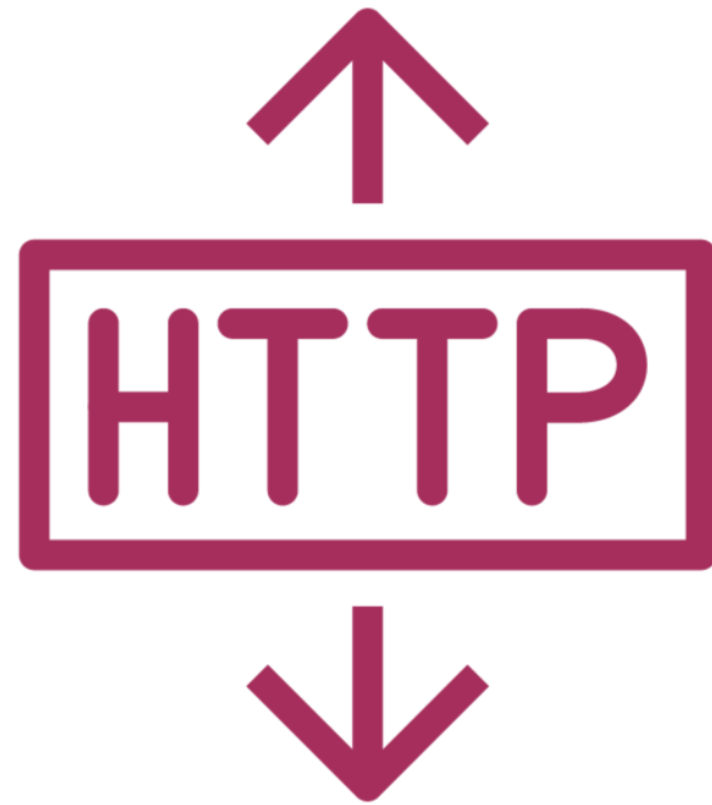


**Undermines stateless nature of
JWTs**

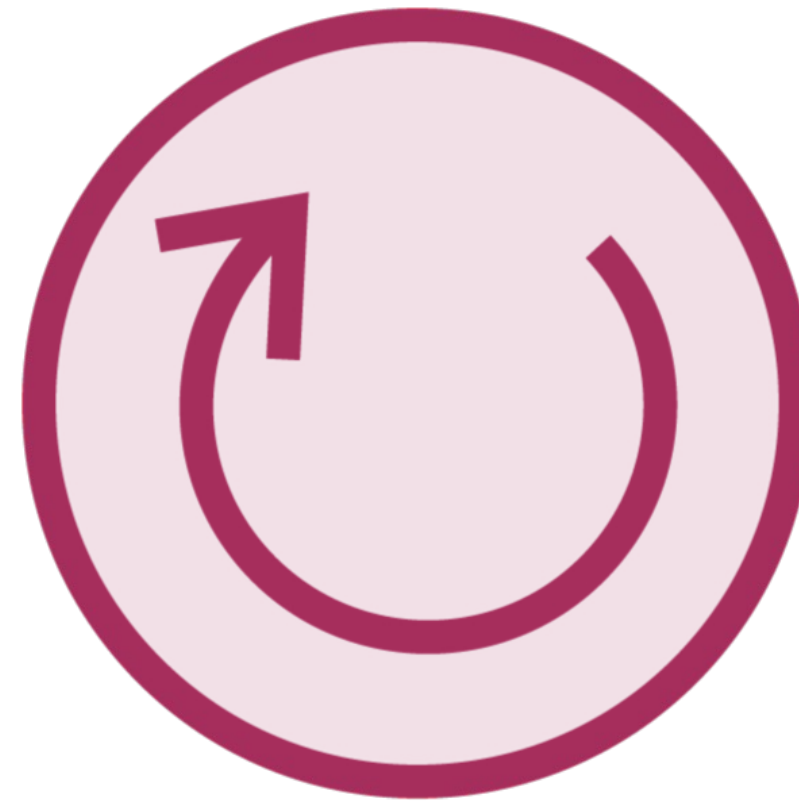
No Application or Permission Data



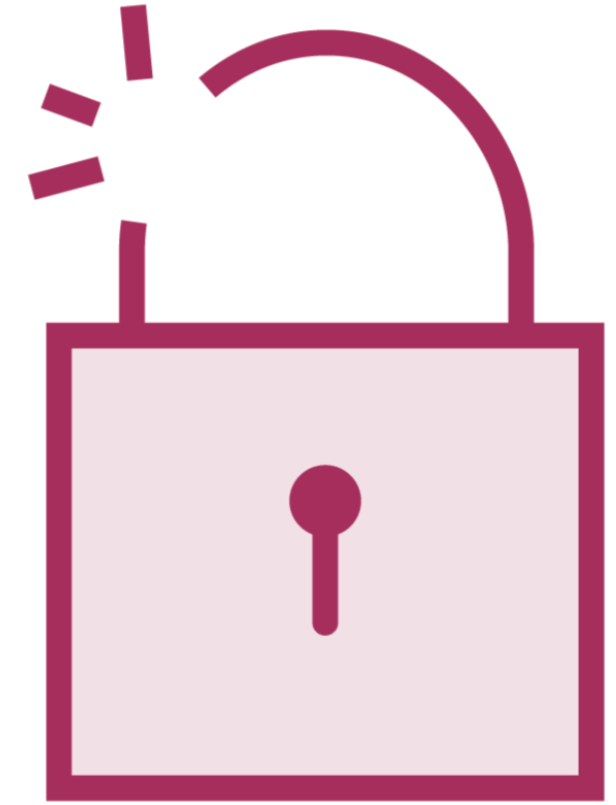
**Keep your JWTs
small**



**Can easily hit
header size
limits**



**This data
changes all the
time**



**If not updated,
this data can
cause security
incidents**

JWTs with Permissions

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "iat": 1656060014,  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "name": "Scott Brady",  
  "subscriptions": [  
    { "id": 1, "account": 123, "type": "premium", "join_date": 1656010314 },  
    { "id": 2, "account": 321, "type": "joint", "join_date": 16560003632 }  
  ],  
  "roles": [  
    "admin",  
    "super_user",  
    "can_read_employee_records",  
    "can_write_employee_records"  
  ]  
}
```



JWTs without Permissions

```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "exp": 1656060314,  
  "iat": 1656060014,  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "name": "Scott Brady",  
  "department": "engineering",  
  "role": "engineering_manager"  
}
```



The JOSE Standards



JavaScript Object Signing and Encryption (JOSE)

**JSON Web Token
(JWT)**

RFC 7519

**JSON Web Signature
(JWS)**

RFC 7515

**JSON Web Encryption
(JWE)**

RFC 7516

**JSON Web Key
(JWK)**

RFC 7517

**JSON Web Algorithms
(JWA)**

RFC 7518



```
{  
  "iss": "https://idp.example.com",  
  "aud": "https://api.example.com",  
  "sub": "b3bfa8d95a17468292a87995e36552dd",  
  "exp": 1656060314,  
  "iat": 1656060014  
}
```

JWT (JSON Web Token)

Core token structure

Standard claim types

eyJ0eXAiOiJKV1QiLCJhbGciOiJFUzI1NiIsImtpZCI6IjE1NmE0YzkwZTQ1OTk0YjJkMTYzMzU0ODBlNzB1NTJhIn0.eyJpc3MiOiJodHRwczovL2lkC5leGFtcGxlLmNvbSIsImF1ZCI6Imh0dHBzOi8vYXBpLmV4YW1wbGUuY29tIiwic3ViIjojYjNiZmE4ZDk1YTE3NDY4MjkyYTg3OTk1ZTM2NTUyZGQiLCJleHAiOjE2NTYwNjAzMTQsIm1hdCI6MTY1NjA2MDAxNH0.n2BxISr_0w800gmdSsS4Xvc9sY22x17-W7EKzCSmItmiDwRZZSCud2cprLVQx67wNNji oDlUeBKnLQVix7kYYA

JWS (JSON Web Signature)

How to sign JWTs

How to create and validate JWT signatures

eyJhbGciOiJSU0EtT0FFUCIsImVuYyI6IkpEYNTZDQkMtSFM1MTIiLCJraWQiOiI1MDNiYTkyMDQxZjE0YTRkYjVmOGI5ZDc4NzM3OTQwZSI6IkpXVCIsImN0eSI6IkpXVCJ9.pTHHed-IGYf_7mleQEY010h7gb10HMEBcjz254HJTyX3h5SCINxJ546mlabX05zhxDFXDlPnDkz4qzuR9671ioD8Y2bAwj2zdHNuzcheQK1LgitcXfeUCPRRUaCdm6me_oBTX7a6eJTn_QaVhBpYXDHJ35CJ0nMZmQLz3GrkitrI2UTn59I0-X0gRNscByEtWZlbXBZhYljaaxxmJ2cexN79YYc1df3zg81uJn0XgPVwWjys6n2bAWi_g7L8GRbrIKN3L77q9KGLM_6y-Phv-eyshi3C5z6zD7MspZ12XS_GTEHfF1FIPkk_jwdJVylrJ864v1nTxQ6TI16H4gS2kg.Bfb0EauIdjxAYBNYMA5MCA.zmXKL2zMCa1xPK4CinQu4GQBBAvfkP-XlPS0WpMvCQhJ0WePRiWXsNxzpyoe8GiAg8j8Je8oxkV7P_ETlr7pmn1uu90p9bX3Lm7ThmULKB3pr6nt8Zm7wIpIV4SPs9D7b8rhF0WKg7_lubrNtFM2Acib0sejZa2QFkU5jYBCq2Z19zBZ90njIXcxqUrT_lvVi7ULIfdVALcRwHtFB3vArEv2jB42bXYAd7kfQyfAjWSVJr6jG0YuBIKIT5PnW1x0ZZWvxph0HpWEQiW8TpjrYbDwycsDM2dbi-4_kZsMyA0zz7oIxQqaNB_M07HR2hHQfo_73WToGh5Gu79HEusZmTCycoNJDWnsSushuTVdIBafGSoTaYqRyxhnIbF2GHjLguMGSDpvt0HmMKXborizoYQSdH5MpprL8pkAvYIJNRwkU353bJlhPo5foQZ0G-pnAVT036QU2cEkE3s-sU5jRJVLsmN2v2APo2fbyyy11H80Gj43c4El0xQoS37QKpwL7m1xJR3QJ_9tiFZJzYPmd0A2025U7v2HPykyLuvCoakNrh1FYr5vTLjZK11GUih3DQDGbGQfhQu00A186bMBsR9rNnAPL4V0oybdqZpAQDTFsX-fh2p7jF8xP50DRN4NqwSPFLjGDAX1g_nFPuYpWJKkFgYTsJyzkerZGSB17nVCkVbb0swR00cbFo_gZumb37e7YsLijDw1bfPmGQqw37DA_oKqSHP3aqDIaxfSQM.12Y11L2b45S9GY7aak015IVKZW-_J-mL1BoHS1IFpX0

JWE (JSON Web Encryption)

How to encrypt JWTs

Different format to JWS

```
{
  "crv": "P-256",
  "alg": "ES256",
  "use": "sig",
  "kty": "EC",
  "kid": "04060a5f378d8627a21396e553665bb6",
  "x": "1s2v39Iih3jT-9qMz-AFRWaD6pKGvTGiFy_9BNrX3no",
  "y": "nxbQzScsuA22xGw0zsGV00Y7RfRkAW9t1oI1IPqg908"
}
```

JWK (JSON Web Key)

How to describe a signing key as JSON

"alg" Param Value	Digital Signature or MAC Algorithm	Implementation Requirements
HS256	HMAC using SHA-256	Required
RS256	RSASSA-PKCS1-v1_5 using SHA-256	Recommended
ES256	ECDSA using P-256 and SHA-256	Recommended+
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256	Optional

JWA (JSON Web Algorithms)

Describes cryptographic identifiers used through JOSE

IANA Registry

Algorithm Name 	Algorithm Description 	Algorithm Usage Location(s) 	JOSE Implementation Requirements 	Change Controller 	Reference 
HS256	HMAC using SHA-256	alg	Required	[IESG]	[RFC7518, Section 3.2]
HS384	HMAC using SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.2]
HS512	HMAC using SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.2]
RS256	RSASSA-PKCS1-v1_5 using SHA-256	alg	Recommended	[IESG]	[RFC7518, Section 3.3]
RS384	RSASSA-PKCS1-v1_5 using SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.3]
RS512	RSASSA-PKCS1-v1_5 using SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.3]
ES256	ECDSA using P-256 and SHA-256	alg	Recommended+	[IESG]	[RFC7518, Section 3.4]
ES384	ECDSA using P-384 and SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.4]
ES512	ECDSA using P-521 and SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.4]
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256	alg	Optional	[IESG]	[RFC7518, Section 3.5]
PS384	RSASSA-PSS using SHA-384 and MGF1 with SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.5]
PS512	RSASSA-PSS using SHA-512 and MGF1 with SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.5]
none	No digital signature or MAC performed	alg	Optional	[IESG]	[RFC7518, Section 3.6]

<https://www.iana.org/assignments/jose/jose.xhtml>



JavaScript Object Signing and Encryption (JOSE)

**JSON Web Token
(JWT)**

RFC 7519

**JSON Web Signature
(JWS)**

RFC 7515

**JSON Web Encryption
(JWE)**

RFC 7516

**JSON Web Key
(JWK)**

RFC 7517

**JSON Web Algorithms
(JWA)**

RFC 7518



JWS JSON Serialization

```
{  
  "payload": "eyJpc3MiOiJqb2UiLA0KICJleHAiOiJlZjMDA4MTkzODAsDQogImh0dHA6Ly9leGFtcGxlLmNvbS9pc19yb290Ijp0cnVlfQ",  
  "protected": "eyJhbGciOiJFUzI1NiJ9",  
  "header": {"kid": "e9bc097a-ce51-4036-9562-d2ade882db0d"},  
  "signature": "DtEhU3ljbEg8L38VWAfUAq0yKAM6-Xx-F4GawxaepmXFCgfTjDxw5djxLa8ISlSApmWQxfKTUJqPP3-Kg6NU1Q"  
}
```



Summary



What is a JWT?

What are JWTs used for?

How do you use a JWT?

When should you use a JWT?

What is JOSE?





More Information

Getting Started with OAuth 2

Scott Brady

